ЛАБОРАТОРНАЯ РАБОТА №4

Модуль 2. Часть 1. Базовые типы данных – строки

Цель работы: формирование опыта работы с базовым типом данных – строки.

Строки допускается использовать для представления почти всего, что может кодироваться как текст или байты.

Строки являются первым представителем более крупного класса объектов, называемых последовательностями. Операции над строками, будут работать с другими типами последовательностей, которые мы рассмотрим позже, такими как списки и кортежи.

Строки Python относятся к категории **неизменяемых последовательностей**, а это означает, что содержащиеся в них символы имеют позиционный порядок слева направо и не могут быть модифицированы на месте.

Строки поддерживают операции выражений, такие как конкатенация (объединение строк), нарезание (срезы, извлечение частей), индексация (извлечение по смещению) и т.д.

Помимо операций выражений Python также предоставляет набор строковых методов, которые реализуют общие задачи, специфичные для строк, и модули для решения более сложных задач вроде сопоставления с образцом.

Распространенные строковые литералы и операции

Операция	Описание
`S = ''`	Пустая строка
`S = "spam's"`	Двойные кавычки; то же самое, что и одинарные
`S = 's\np\ta\x00m'`	Управляющие последовательности
`S = """...много строк..."""`	Блочные строки в утроенных кавычках
`S = r'\temp\spam'`	Неформатированные строки (без управляющих символов)
`B = b'sp\xc4m'`	Байтовые строки в Python 2.6, 2.7 и 3.X (см. главы 4 и 37)
`U = u'sp\u00c4m'`	Строки Unicode в Python 2.X и 3.3+ (см. главы 4 и 37)
`S1 + S2` `S*3`	Конкатенация, повторение

`S[i]`
`S[i:j]`
`len(S)`
	Индекс, срез, длина

`"a %s parrot" % kind`	Выражение форматирования строки
`"a {0} parrot"` `.format(kind)S.find('pa')`	Метод форматирования строки в Python 2.6, 2.7 и 3.X

Базовые строчные операции

Мы можем выполнять конкатенацию строк, используя операцию +, и повторять их с применением операции *:

```
>>> len('abc')          # Длина: количество элементов
3
>>> 'abc' + 'def'       # Конкатенация: новая строка
'abcdef'
>>> 'Ni! ' * 4          # Повторение: подобно "Ni!" + "Ni!" + ...
'Ni!Ni!Ni!Ni!'
```

Встроенная функция len возвращает длину строки (или любого другого объекта, имеющего длину). Формально сложение двух строк посредством операции + создает новый строковый объект с объединенным содержимым ее операндов, а повторение с помощью операции * подобно добавлению строки к самой себе несколько раз. В обоих случаях Python позволяет создавать строки произвольных размеров.

Например, для вывода 80 символов минуса мы можем сами посчитать до 80 или поручить это Python:

```
>>> print(' -----...и еще... -----')      # 80 символов трудный способ
>>> print('-' * 80)                       # 80 символов легкий способ
```

Обратите внимание, что здесь уже работает **перегрузка операций**: мы используем те же самые операции + и *, которые выполняют сложение и умножение, когда применяются к числам. Язык Python делает правильную операцию, поскольку ему известны типы объектов в операндах.

Индексация и нарезание

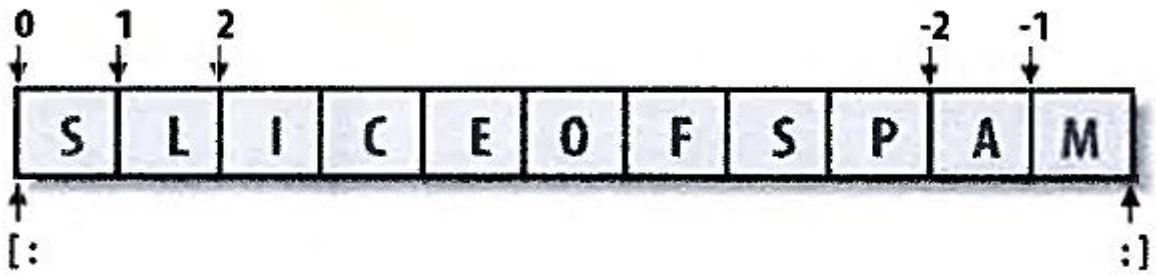
Поскольку строки определяются как упорядоченные коллекции символов, мы можем получать доступ к их компонентам по позиции.

В Python символы из строки извлекаются посредством индексации — **предоставления числового смещения** желаемого компонента в квадратных скобках после строки. Результатом будет односимвольная строка в указанной позиции.

Смещения в Python начинаются с 0 и заканчиваются величиной на единицу меньше длины строки.

Python также позволяет извлекать элементы из последовательностей, таких как строки, **с применением отрицательных смещений**, которые представляют собой отсчет в обратном направлении с конца.

```
>>> s = 'spam'
>>> S[0] , S[-2]              # Индексация с начала или с конца
('s', 'a')
>>> S[1:3], S[1:], S[:-1]     # Нарезание: извлечение сегмента
('pa', 'pam', 'spa')
```



В первой команде определяется четырехсимвольная строка и присваивается имени S. В следующей команде она индексируется двумя путями: S [0] извлекает элемент по смещению 0 слева — односимвольную строку ’s’; S [-2] получает элемент по смещению 2 с конца.

*1. Индексация (**S[±]**) извлекает компоненты по смещениям:*

- первый элемент находится по смещению 0;
- отрицательные смещения означают отсчитывание с конца или справа;
- S [0] извлекает первый элемент;
- S [-2] извлекает второй элемент с конца (подобно S [len (S) -2]).

*2. Нарезание (**S[i : j]**) извлекает непрерывные сегменты последовательности:*

- верхняя граница является исключающей;
- если границы не указаны, тогда по умолчанию для них принимаются 0 и длина последовательности;
- S [1:3] извлекает элементы, которые начинаются со смещения 1 и заканчиваются по смещению 3, не включая его;
- S [1:] извлекает элементы, начиная со смещения 1 и до конца;
- S [: 3] извлекает элементы, которые начинаются со смещения которые начинаются со смещения 0 и заканчиваются по смещению 3, не включая его;
- S [:-1] извлекает элементы, начиная со смещения 0 и заканчивая последним элементом, но не включая его;
- S [:] извлекает элементы, начиная со смещения 0 и до конца — создает копию S.

*3. Расширенное нарезание (**S [i : j : k]**) принимает шаг (или страйд - большой шаг) к, который по умолчанию равен +1:*

- позволяет пропускать элементы и менять порядок на противоположный, что объясняется далее.

Для выражений срезов имеется поддержка необязательного третьего индекса, используемого в качестве шага. Шаг добавляется к индексу каждого извлеченного элемента. Теперь полная форма среза выглядит

как X [I : J : K] и означает “извлечь все элементы в X, начиная со смещения I и заканчивая смещением J-1, с шагом K”. Третий предел, K, по умолчанию устанавливается в +1 и поэтому все элементы в срезе обычно извлекаются слева направо. Однако если вы укажете явное значение, то сможете применять третий предел для пропуска элементов или смены порядка их следования на противоположный.

Например, срез X [1 : 10 : 2] будет извлекать каждый второй, элемент из X в рамках смещений 1-9, т.е. он накопит элементы по смещениям 1, 3, 5, 7 и 9. Как обычно, первый и второй пределы по умолчанию принимаются равными 0 и длине последовательности соответственно, так что X [: : 2] получает каждый второй элемент с начала и до конца последовательности:

```
>>> S = ' abcdefghijklmnop'
>>> S [1:10:2]          # Пропуск элементов
’bdfhj'
>>> S[: :2]
’acegikmo’
```

Можно также использовать отрицательный страйд для накапливания элементов в противоположном порядке.

```
>>> S= ’hello’
>>> S[: : -1]           # Смена порядка, следования элементов на
                        # противоположный
’olleh’
```

При отрицательном шаге смысл первых двух границ по существу изменяется на противоположный. Таким образом, срез S [5 : 1 : -1] извлекает элементы со второго по пятый в обратном порядке (результат содержит элементы по смещениям 5, 4, 3 и 2):

```
>>> S = 'abcedfg'
>>> S[5:1:-1]           # Смысл границ изменяется
'fdec'
```

Инструменты преобразования строк

Python не разрешает смешивать числа и строки в выражениях, мы не можем складывать число и строку в Python, даже если эта строка выглядит похожей на число (т.е. содержит только цифры):

```
# Python 3.X
>>> „42" + 1
TypeError: Can’t convert ’int’ object to str implicitly
```

Ошибка типа: нельзя преобразовывать объект int в str неявно.

```
>>> int("42"), str(42)     # Преобразование из строки и в строку
(42, ’42')
```

Функция int преобразует строку в число, а функция str преобразует число в его строковое представление (по существу в то, как число выглядит, когда выводится посредством print).

```
>>> s=‘42’
>>> int(s) + 1              # Принудительно применить сложение
43
>>> s + str(1)              # Принудительно применить конкатенацию
' 421'
```

Похожие встроенные функции поддерживают преобразования чисел с плавающей точкой в строки и обратно:

```
>>> str(3.1415), float("1.5")
( '3.1415', 1.5)
>>> text = "1.234E-10"
>>> float (text)
1.234e-10
```

Передавая символ встроенной функции ord, которая возвратит действительное двоичное значение, используемое для представления соответствующего символа в памяти.

Функция chr выполняет обратное преобразование, принимая целочисленный код и возвращая надлежащий символ:

```
>>> ord( 's')
115
>>> chr (115)
' s'
```

Например, чтобы перейти к следующему символу, его нужно преобразовать и выполнить математическое действие с целым числом:

```
>>> s = ’5’
>>> s = chr(ord(s) + 1)
>>> s
’6’
>>> s = chr(ord(s) + 1)
>>> s
'7'
```

Термин “**неизменяемая последовательность**” означает, что мы не можем изменять строку на месте — скажем, присваивая по индексу:

```
>>> s = ’spam’
>>> s[0] = ’x’              # Возникает ошибка!
TypeError: ’str’ object does not support item assignment
```

Ошибка типа: объект str не поддерживает присваивание в отношении элементов.

Тогда каким же образом модифицировать текстовую информацию в Python? Чтобы изменить строку, обычно необходимо создать и присвоить новую строку с применением таких инструментов, как конкатенация и нарезание, и затем при желании присвоить результат первоначальному имени строки:

```
>>> s = s + ’SPAM!’          # Чтобы изменить строку, нужно
                             # создать новую
>>> s
’spamSPAM!'
>>> s = s[: 4] + ’Burger’ + s[-1]
>>> s
’spamBurger!’
```

В первом примере подстрока добавляется в конец S посредством конкатенации. На самом деле здесь создается новая строка и присваивается переменной S, но мы можем думать об этом как об “изменении” исходной строки.

Задачи для самостоятельного выполнения

1. Дан символ C. Вывести его код (то есть номер в кодовой таблице).
2. Дано целое число N ($1 \le N \le 26$). Вывести N последних строчных (то есть маленьких) букв латинского алфавита в обратном порядке (начиная с буквы «z»).
3. Дан символ C. Вывести два символа, первый из которых предшествует символу C в кодовой таблице, а второй следует за символом C.
4. Дано целое число N (> 0) и символ C. Вывести строку длины N, которая состоит из символов C.
5. Дана непустая строка S. Вывести строку, содержащую символы строки S, между которыми вставлено по одному пробелу.
6. Дана непустая строка S и целое число N (> 0). Вывести строку, содержащую символы строки S, между которыми вставлено по N символов «*» (звездочка).

Строки – индексация. Получение срезов строк

7. Дана непустая строка. Вывести коды ее первого и последнего символа.
8. Дана строка. Вывести строку, содержащую те же символы, но расположенные в обратном порядке.
9. Дана строка длиной более 10 символов. Создать ее копию, через использование среза.
10. Дана строка длиной более 10 символов. Получить из нее 5-тый элемент строки.
11. Дана строка длиной более 10 символов. Получить из нее последний элемент строки.
12. Дана строка длиной более 10 символов. Получить из нее срез из первых 5 символов.
13. Дана строка длиной более 10 символов. Получить из нее срез из последних 5 символов.
14. Дана строка длиной более 10 символов. Получить из нее срез из 5 символов, начиная с 4 символа строки.
15. Дана строка длиной более 10 символов. Получить из нее срез из 5 символов, начиная с 4 символа от конца строки.
16. Дана строка длиной более 10 символов. Получить из нее каждый второй элемент строки.
17. Дана строка длиной более 10 символов. Получить из нее каждый второй элемент строки, расположенные в обратном порядке.
18. Дана строка длиной более 10 символов. Получить из нее срез из 5 символов, начиная с 4 символа от конца строки, идущие в обратном порядке.

Посимвольный анализ и преобразование строк. Строки и числа

19. Дана строка. Подсчитать количество содержащихся в ней цифр.
20. Дана строка. Подсчитать количество содержащихся в ней прописных латинских букв.
21. Дана строка. Преобразовать в ней все прописные латинские буквы в строчные.
22. Дана строка. Преобразовать в ней все строчные буквы (как латинские, так и русские) в прописные, а прописные – в строчные.
23. Дана строка, изображающая целое положительное число. Вывести сумму цифр этого числа.
24. Дана строка, изображающая целое положительное число. Вывести сумму цифр этого числа.
25. Дана строка, изображающая двоичную запись целого положительного числа. Вывести строку, изображающую десятичную запись этого же числа.

Дополнительные задания по теме

1. Дано целое число N ($32 \le N \le 126$). Вывести символ с кодом, равным N.
2. Дано целое число N ($1 \le N \le 26$). Вывести N первых прописных (то есть заглавных) букв латинского алфавита.
3. Дано целое число N ($1 \le N \le 26$). Вывести N последних строчных (то есть маленьких) букв латинского алфавита в обратном порядке (начиная с буквы «z»).
4. Дана строка. Преобразовать в ней все строчные буквы (как латинские, так и русские) в прописные.
5. Дано целое положительное число. Вывести символы, изображающие цифры этого числа (в порядке справа налево).
6. Дан символ C и строки S, S0. После каждого вхождения символа C в строку S вставить строку S0.
7. Дана строка, изображающая десятичную запись целого положительного числа. Вывести строку, изображающую двоичную запись этого же числа.
8. Дана строка длиной более 10 символов. Получить из нее 9-тый элемент строки.
9. Дана строка длиной более 10 символов. Получить из нее предпоследний элемент строки.
10. Дана строка длиной более 10 символов. Получить из нее срез из первых 9 символов.
11. Дана строка длиной более 10 символов. Получить из нее срез из последних 6 символов.
12. Дана строка длиной более 10 символов. Получить из нее срез из 4 символов, начиная с 3 символа строки.
13. Дана строка длиной более 10 символов. Получить из нее срез из 4 символов, начиная с 3 символа от конца строки.
14. Дана строка длиной более 10 символов. Получить из нее каждый третий элемент строки.
15. Дана строка длиной более 10 символов. Получить из нее каждый третий элемент строки, расположенные в обратном порядке.
16. Дана строка длиной более 10 символов. Получить из нее срез из 6 символов, начиная с 3 символа от конца строки, идущие в обратном порядке.